

oto Command Routing Guide

A full card for every `/oto-*` command: when to trigger it, what it produces, whether it's **Opus** (thinking) or **Codex** (execution) work, runtime notes, gotchas, and what comes next. oto unifies GSD's spec-driven workflow with Superpowers' capabilities behind one `/oto-*` surface across Claude Code, Codex, and Gemini CLI.

The plan is the leverage point. The pattern is simple: **Opus for thinking** — planning, design, diagnosis, judgment — and **Codex (GPT) for execution**, dashboards, and summarization. A strong model writing a clean, spec-style plan that a hard executor grinds out beats either model trying to do both. Keep plans spec-style — numbered steps, explicit paths, acceptance criteria — and the Opus → Codex handoff just works.

Legend

Each card has a left-edge bar in its engine color, the command name, and an engine pill. Fields: **WHEN** triggers, **PRODUCES** artifacts, **WHY** rationale, **HEADS UP** gotchas, **NEXT** what follows, **RUNTIME** where it runs best.

- OPUS** Planning, design, diagnosis, judgment, voice-sensitive output.
- CODEX** Execution, generation, dashboards, summarization, operational coordination. GPT via Codex.
- EITHER** Mechanical or routing commands. Cost or speed decides the engine.
- SPLIT** Different sub-tasks want different engines. The card explains.

How Codex enters oto. By default oto runs in Claude Code. To put the **CODEX** commands on GPT, either run oto inside a Codex session or turn on cross-AI execution in `/oto-settings-advanced`. `/oto-review` and `/oto-plan-review-convergence` already reach Codex, Gemini, and Claude directly. `/oto-set-profile` tunes oto's Claude-side agents separately — that's a cost dial, not the Opus-vs-Codex routing this guide is about.

Contents

01	Core Workflow	<i>the milestone loop, every phase runs through these</i>
02	Phase Management	<i>roadmap edits, coverage, specialized phase types</i>
03	Code Quality	<i>cross-AI review, PR hygiene, fixes, docs</i>
04	Diagnostics & Debug	<i>forensics, codebase mapping, debugging</i>
05	Ideation & Discovery	<i>explore, spike, sketch — the fuzzy front end</i>
06	Utility	<i>quick tasks, todos, backlog, captures</i>
07	Navigation & Workstreams	<i>session state, parallel tracks, workspaces</i>
08	Project Setup & Migration	<i>onboarding docs, external plans, GSD migration</i>
09	Config & Update	<i>profiles, settings, framework maintenance</i>

01 CORE WORKFLOW

The milestone loop. Every phase runs through some subset of these. Use `/oto-next` when you forget what comes after what.

/oto-new-project

OPUS

Initialize a new project with deep context gathering and **PROJECT.md**.

- WHEN** Starting a brand-new oto-managed project — no existing **.oto/PROJECT.md**. Run with **--auto** to extract from an existing PRD; otherwise the interactive flow walks questioning → research → requirements → roadmap.
- PRODUCES** **PROJECT.md** (mission, users, constraints) · **REQUIREMENTS.md** · **ROADMAP.md** · **STATE.md**, **config.json**, and the **.oto/**scaffold.
- WHY** Pure thinking work. Setting the project's center of gravity wrong here costs you for the entire milestone — every downstream phase reads **PROJECT.md**. Opus's product judgment compounds.
- HEADS UP** Don't rush this with **--auto** on a half-baked PRD. Garbage in, milestone-long garbage out. For an existing repo, run **/oto-map-codebase** or **/oto-ingest-docs** first.
- NEXT** **/oto-spec-phase 1** or **/oto-discuss-phase 1** to start the first phase.
- RUNTIME** any runtime

/oto-spec-phase

OPUS

Socratic spec refinement — lock **WHAT** a phase delivers before **HOW**.

- WHEN** Before **/oto-discuss-phase** on any phase where the requirement itself is fuzzy. Runs an interview loop (up to 6 rounds) scoring ambiguity across 4 dimensions; gates on ambiguity at or below 0.20 before writing the spec.
- PRODUCES** {phase} **-SPEC.md** — falsifiable requirements that lock what/why. **discuss-phase** picks it up automatically next run.
- WHY** Requirements judgment is the highest-leverage thinking in the loop, and the ambiguity scoring is only as good as the model surfacing the gaps. Opus territory.
- HEADS UP** Skip for phases whose deliverable is already crisp — you'll just confirm what you already know. Best on net-new capability where 'done' is debatable.
- NEXT** **/oto-discuss-phase N** once the spec is committed.
- RUNTIME** any runtime

/oto-discuss-phase

OPUS

Capture implementation decisions and surface trade-offs before planning.

- WHEN** Before **/oto-plan-phase** on any phase with unknowns, design ambiguity, or competing approaches. Skip only when the work is so mechanical it has no real decisions (e.g. dependency bumps). **--all** discusses every gray area; **--auto** picks defaults silently; **--power** batches questions into a file UI.
- PRODUCES** {phase} **-CONTEXT.md** (decisions captured) · a discussion log as audit trail.
- WHY** The whole point is to surface ambiguity, not paper over it — that wants Opus's product eye. A tendency toward tidy summaries is a liability here.
- HEADS UP** **--auto** picks recommended defaults silently; use it only when you genuinely don't care about the answers. **--analyze** adds trade-off depth worth the time on real decisions.
- NEXT** **/oto-ui-phase** if any frontend is involved, otherwise straight to **/oto-plan-phase**.
- RUNTIME** any runtime

/oto-ui-phase

OPUS

Lock the UI design contract — spacing, type, color, copy — before planning.

- WHEN** After discuss-phase, before plan-phase, for any phase with frontend work. Skip for pure backend. Locks the contract once so five components don't end up with five slightly different button styles.
- PRODUCES** {phase} **-UI-SPEC.md** — design tokens, component patterns, copy tone, registry-safety rules.
- WHY** Design coherence and spatial composition are judgment calls. A 'technically correct but visually random' UI is exactly the failure mode Opus avoids.
- NEXT** **/oto-plan-phase** — now the planner has both context and a design contract.
- RUNTIME** any runtime

/oto-plan-phase

OPUS

Research, plan, and verify a phase. The single highest-leverage command.

- WHEN** After discuss-phase (and ui-phase if applicable). Or with **--prd** file.md to skip discussion when a PRD already exists. Use **--reviews** to replan after **/oto-review**. **--tdd** front-loads tests.
- PRODUCES** {phase} **-RESEARCH.md** · {phase} **-N-PLAN.md** (one per atomic plan, each sized to a fraction of a context window) · validation notes.
- WHY** The leverage point. A clean spec-style plan executes far better than a narrative one — and Opus writes the better plan. Push it toward 'exactly what to do, in order, with acceptance criteria' so whichever engine executes can follow it hard.
- HEADS UP** Use **--skip-research** if you know the domain cold. Don't skip it where techniques move fast and a stale assumption would derail the wave.
- NEXT** **/oto-review --phase N --all** to cross-pollinate, then **/oto-execute-phase**.
- RUNTIME** any runtime

/oto-execute-phase

CODEX

Execute all plans in a phase with wave-based parallelization.

- WHEN** After plans exist. Run with `--wave N` for a checkpoint between waves, or without to run all waves to completion. Independent plans run in parallel. `--tdd` enforces red-green-refactor.
- PRODUCES** Per-plan {phase} `-N-SUMMARY.md` · atomic git commits (one per task) · a verification artifact when the phase completes.
- WHY** Sustained execution across a long thread — writing code, deleting code, resisting ineffectual patching — is Codex/GPT's sweet spot, not a place to spend judgment-tier reasoning. The plan already carries the judgment; the executor just grinds it out.
- HEADS UP** To put execution on GPT, enable cross-AI execution in `/oto-settings-advanced` or run oto inside a Codex session, then merge back. Otherwise the wave runs in the active runtime.
- NEXT** `/oto-verify-work N` — back to Opus for fresh-eyes UAT.
- RUNTIME** Codex / ChatGPT

/oto-verify-work

OPUS

Conversational user-acceptance testing with auto-diagnosis.

- WHEN** Immediately after execute-phase. Don't skip it — it's the gate that catches what tests don't. If issues surface, oto generates fix plans you can run back through plan-phase.
- PRODUCES** {phase} `-UAT.md` · fix plans if issues surface · `STATE.md` marked verified or blocked.
- WHY** Fresh eyes that weren't in the execution loop. UAT is judgment, not execution — letting the executor verify its own work inherits the same blind spots that produced it.
- NEXT** `/oto-ui-review` if frontend work was done · `/oto-ship` if green · `/oto-plan-phase` if fix plans were created.
- RUNTIME** any runtime

/oto-ui-review

OPUS

Retroactive 6-pillar visual audit of implemented frontend.

- WHEN** After execute-phase on any phase that produced frontend code. Works standalone too — useful for a quick visual audit of UI someone else built.
- PRODUCES** {phase} `-UI-REVIEW.md` · screenshots under .oto/ for visual reference.
- WHY** Visual judgment and design-coherence assessment — the eye that catches 'each piece is fine but the whole feels random.' Opus.
- NEXT** Fix plans via `/oto-plan-phase` if issues surface, otherwise `/oto-ship`.
- RUNTIME** any runtime

/oto-ship

CODEX

Create a PR, run review, and prepare for merge after verification passes.

- WHEN** After verify-work passes. Requires the gh CLI authenticated. Pass a phase or milestone; use `--draft` for early review.
- PRODUCES** A GitHub PR whose body synthesizes phase goal, plan summaries, requirements covered, verification status, and key decisions. `STATE.md` updated.
- WHY** Operational synthesis from artifacts that already contain the judgment. Codex assembles the PR — no need to spend Opus on it.
- NEXT** `/oto-pr-branch` first if you want to filter .oto/ commits out of the reviewer's diff.
- RUNTIME** any runtime

/oto-next

EITHER

Automatically advance to the next logical step in the workflow.

- WHEN** When you return after a break and don't remember where you left off, or when you trust oto to pick the next move. It reads `STATE.md` and `ROADMAP.md` and dispatches.
- PRODUCES** Whatever the chosen sub-command produces — it's a router, not a worker.
- WHY** Pure routing logic. The engine is whatever the dispatched command needs.
- NEXT** Whichever command it dispatched.
- RUNTIME** any runtime

/oto-session-report

CODEX

Generate a session report with token estimates, work summary, and outcomes.

- WHEN** End of a working session — before stepping away, switching contexts, or handing off to a collaborator.
- PRODUCES** `.oto/reports/` session report with commits, plans executed, phases progressed, blockers, decisions, token usage, and next steps.
- WHY** Turning messy session state into an orderly record is summarization — Codex's strong suit, and not worth Opus tokens.
- NEXT** `/oto-pause-work` if stopping mid-phase, otherwise resume normally.
- RUNTIME** any runtime

/oto-audit-uat

CODEX

Cross-phase audit of all outstanding UAT and verification items.

- WHEN** When you've executed multiple phases and need to know what's still incomplete across all of them — especially before a milestone-completion attempt or after a long pause.
- PRODUCES** A categorized audit report with a human test plan for items needing manual verification.
- WHY** Tabular synthesis across artifacts — orderly aggregation, exactly what Codex is good at.
- NEXT** `/oto-audit-fix` to auto-clear fixable items, or `/oto-plan-phase` for the rest.
- RUNTIME** any runtime

/oto-audit-milestone

OPUS

Audit milestone completion against original intent before archiving.

WHEN	After all phases in the milestone are executed and verified, before complete-milestone. The gate that asks 'did we actually ship what we said we would?'
PRODUCES	An audit report with gap analysis — what's done, what's missing, what drifted.
WHY	Higher-stakes than audit-uat: decision support, not summarization. Wants Opus's sharper read on whether claimed completion matches actual completion.
NEXT	<code>/oto-plan-milestone-gaps</code> if gaps exist, otherwise <code>/oto-complete-milestone</code> .
RUNTIME	any runtime

/oto-complete-milestone

CODEX

Archive a completed milestone and prepare for the next version.

WHEN	After audit-milestone shows clean. This is ceremony — the decision is already made; now you're recording it.
PRODUCES	A milestones entry with summary · a git tag for the release.
WHY	Bookkeeping plus a little summary. Codex handles the archive and tag; the judgment happened upstream.
NEXT	<code>/oto-milestone-summary</code> for team-readable docs · <code>/oto-new-milestone</code> to start the next cycle.
RUNTIME	any runtime

/oto-milestone-summary

CODEX

Comprehensive project summary from milestone artifacts for onboarding.

WHEN	After complete-milestone, or anytime you need to brief someone new — contractor onboarding, an advisor sync, a status update. Pass a version to summarize a historical milestone.
PRODUCES	<code>.oto/reports/</code> milestone summary: overview, architecture decisions, phase-by-phase breakdown, trade-offs, requirements coverage, tech debt, and a getting-started guide.
WHY	Structured prose for a broad audience with clean idea progression — the readable-narrative sweet spot. Codex produces it; Opus's punchier voice would be wasted here.
NEXT	Onboarding · status updates · <code>/oto-new-milestone</code> .
RUNTIME	any runtime

/oto-new-milestone

OPUS

Start the next milestone cycle — update `PROJECT.md` and re-roadmap.

WHEN	Previous milestone complete and archived. Pass a name like 'v1.1 Notifications' to label it clearly.
PRODUCES	Updated <code>PROJECT.md</code> · fresh <code>REQUIREMENTS.md</code> and <code>ROADMAP.md</code> for the next cycle.
WHY	Center-of-gravity setting, just like new-project. A wrong call here cascades through every phase that follows — Opus.
HEADS UP	Scans <code>.oto/seeds/</code> for forward-looking ideas planted in the previous cycle — make sure you planted them with <code>/oto-plant-seed</code> .
NEXT	<code>/oto-spec-phase 1</code> or <code>/oto-discuss-phase 1</code> in the new milestone.
RUNTIME	any runtime

02 PHASE MANAGEMENT

Modifying the roadmap mid-flight, auditing coverage, and the specialized phase types oto adds on top of GSD.

/oto-add-phase

OPUS

Append a new phase to the end of the current milestone.

- WHEN** Mid-milestone, when scope expands and a new phase is needed. Interactive — describe the phase and oto slots it in.
- PRODUCES** A new phase entry in `ROADMAP.md` with its directory scaffolded.
- WHY** A structural decision about what work happens. Wants product judgment.
- NEXT** `/oto-discuss-phase` on the new phase number.
- RUNTIME** any runtime

/oto-insert-phase

OPUS

Insert urgent work as a decimal phase (e.g. 7.1) between existing phases.

- WHEN** Something urgent surfaced — bug, security issue, blocker — that can't wait. Decimal numbering threads it in without disturbing the existing roadmap.
- PRODUCES** A new phase between N and N+1 with its directory created.
- WHY** Same reasoning as add-phase: a judgment call about priority ordering and dependencies.
- NEXT** `/oto-discuss-phase` 7.1 (or whatever number was assigned).
- RUNTIME** any runtime

/oto-remove-phase

EITHER

Remove a future phase and renumber subsequent phases.

- WHEN** Scope cut — a phase is no longer needed. Only works on future phases; you can't remove already-executed ones.
- PRODUCES** Phase removed, subsequent phases renumbered, `ROADMAP.md` updated.
- WHY** Mechanical renumbering once the decision to remove has been made. The decision wants judgment; the command itself doesn't.
- RUNTIME** any runtime

/oto-edit-phase

EITHER

Edit any field of an existing roadmap phase in place.

- WHEN** When a phase's title, goal, scope, or dependencies need correcting without changing its number or position. Pass `--force` to skip confirmation.
- PRODUCES** The roadmap phase updated in place, number and order preserved.
- WHY** Targeted text surgery on the roadmap. The wording you supply carries the judgment.
- RUNTIME** any runtime

/oto-list-phase-assumptions

OPUS

Surface Claude's assumptions about a phase approach before planning.

WHEN	Before plan-phase on any phase where you suspect an unstated assumption — especially brownfield work where existing patterns might contradict the obvious approach.
PRODUCES	A list of implicit choices, dependencies, and likely approaches the planner would otherwise bake in silently.
WHY	A 'show me what you're thinking' sanity check. Wants the model whose surfaced assumptions are the meaningful ones — Opus.
NEXT	Edit <code>CONTEXT.md</code> if the assumptions are wrong, then <code>/oto-plan-phase</code> .
RUNTIME	any runtime

/oto-analyze-dependencies

CODEX

Analyze the phase dependency graph and suggest 'Depends on' edits.

WHEN	After roadmapping or when phases have grown interdependent and the execution order is no longer obvious. Detects file overlap, semantic dependencies, and data flow between phases.
PRODUCES	Suggested 'Depends on' updates for <code>ROADMAP.md</code> .
WHY	Structured graph analysis over the roadmap and codebase — pattern matching and aggregation more than open-ended judgment.
NEXT	Apply the suggestions, then <code>/oto-execute-phase</code> respects the wave ordering.
RUNTIME	any runtime

/oto-plan-milestone-gaps

OPUS

Create phases to close all gaps from a milestone audit.

WHEN	After audit-milestone identifies gaps. Automates the 'okay, we're missing X, Y, Z — let's plan phases for those' step.
PRODUCES	One new phase per audit gap, ready for the discuss/plan/execute cycle.
WHY	Planning work — same logic as plan-phase.
NEXT	<code>/oto-discuss-phase</code> on each newly-created gap phase.
RUNTIME	any runtime

/oto-research-phase

SPLIT

Standalone deep ecosystem research — usually fold into plan-phase instead.

WHEN	Rare. plan-phase handles research inline most of the time. Use this when you want a research artifact without committing to planning yet — library evaluation, technique exploration.
PRODUCES	{phase} - <code>RESEARCH.md</code> with ecosystem analysis, library options, prior art, and gotchas.
WHY	Opus for diagnostic 'how does this work, where are the gotchas' synthesis. Codex only when you need a structured comparison table across many options. Gemini when the corpus is huge — whole repo plus many papers in one pass.
NEXT	<code>/oto-plan-phase --skip-research</code> since you've already done it.
RUNTIME	Claude Code · Gemini for large corpora

/oto-validate-phase

CODEX

Retroactively audit and fill Nyquist validation gaps for a phase.

- WHEN** After execution, when you suspect test coverage is thin. Audits whether tests actually validate the requirements — not just whether they pass.
- PRODUCES** A coverage audit report with gaps flagged.
- WHY** Coverage audits are checklist work: comparing artifacts against requirements. Codex.
- NEXT** `/oto-add-tests` N to fill in the missing coverage.
- RUNTIME** any runtime

/oto-ai-integration-phase

OPUS

Generate a bounded `AI-SPEC.md` skeleton for AI-powered phases.

- WHEN** When a phase adds an AI feature and you want a design contract for it. v0.1.0 is bounded (per ADR-07): the domain-research step is live; framework selection, AI research, and eval planning print explicit DEFERRED blocks with manual-fill instructions.
- PRODUCES** Domain-research `RESEARCH.md` fragments · an `AI-SPEC.md` skeleton with explicit TODO sections for the deferred steps.
- WHY** Domain research and AI-system design are reasoning-heavy — Opus. The command is honest about which steps are live versus deferred; don't treat the TODO sections as done.
- HEADS UP** Deferred steps are stubs by design. Fill them manually until the eval-tooling agents return in a later version.
- NEXT** `/oto-discuss-phase` or `/oto-plan-phase` once the AI-SPEC is fleshed out.
- RUNTIME** any runtime

/oto-secure-phase

OPUS

Retroactively verify threat mitigations for a completed phase.

- WHEN** After a phase executes, to confirm its threat model was actually mitigated. Handles three states: a `SECURITY.md` exists (audit it), a `PLAN.md` threat model exists (run from artifacts), or the phase wasn't executed (exit with guidance).
- PRODUCES** An updated `SECURITY.md` with mitigation verification.
- WHY** Threat analysis is adversarial reasoning — finding the gap an attacker would. Opus.
- NEXT** Fix plans via `/oto-plan-phase` if mitigations are missing.
- RUNTIME** any runtime

/oto-eval-review

CODEX

Retroactively audit an executed AI phase's evaluation coverage.

- WHEN** After an AI phase executes, to check whether the evaluation strategy from `AI-SPEC.md` was actually implemented. Scores each eval dimension COVERED / PARTIAL / MISSING.
- PRODUCES** `EVAL-REVIEW.md` with a score, verdict, gaps, and a remediation plan.
- WHY** Comparing claimed eval coverage against implemented coverage is a structured audit against a spec — Codex.
- NEXT** Remediation plans through `/oto-plan-phase` for the MISSING dimensions.
- RUNTIME** any runtime

03 CODE QUALITY

Cross-pollinating reviews and PR hygiene — the highest-leverage habit in this guide.

/oto-review --phase N --all

SPLIT

Cross-AI peer review of a phase plan from external AI CLIs.

- WHEN** After plan-phase, before execute-phase. Run it on every plan touching production code. Flags: `--codex`, `--gemini`, `--claude`, `--opencode`, `--qwen`, `--cursor`, or `--all`.
- PRODUCES** {phase} `-REVIEWS.md` consolidating each CLI's feedback — consumed by `/oto-plan-phase --reviews`.
- WHY** The entire value is multi-model: each engine has different blind spots, so a second engine reading your plan catches what the author can't. This is oto's cross-AI spine, not a single-engine call.
- HEADS UP** Don't accept any single review as definitive. One CLI flags missing recent patterns, another design coherence, another execution ambiguity — weigh them together.
- NEXT** `/oto-plan-phase N --reviews` to fold the feedback back into the plan.
- RUNTIME** spawns external AI CLIs

/oto-plan-review-convergence

SPLIT

Cross-AI plan convergence loop — replan with feedback until no HIGH concerns remain.

- WHEN** When a plan is important enough to iterate to convergence rather than a single review pass. Loops review → replan automatically (max 3 cycles, override with `--max-cycles`) across the runtimes you flag.
- PRODUCES** A converged `PLAN.md` plus the per-cycle review trail.
- WHY** Same multi-model logic as `/oto-review`, automated into a loop. Opus does the replanning; the external CLIs (Codex, Gemini, Claude) do the reviewing.
- HEADS UP** Cap the cycles. If concerns aren't converging after a couple of rounds, the disagreement is usually a real design fork that wants a human decision.
- NEXT** `/oto-execute-phase` once HIGH concerns are cleared.
- RUNTIME** spawns external AI CLIs

/oto-code-review

OPUS

Review source files changed during a phase for bugs and quality problems.

- WHEN** After execute-phase, before shipping. Reviews the actual diff — not the plan. Set `--depth=quick|standard|deep`; scope with `--files`.
- PRODUCES** `REVIEW.md` cataloguing bugs, security issues, and code-quality findings by severity.
- WHY** Finding the real bug or security hole in a diff is diagnosis and judgment — Opus. Use `--depth=deep` on security-sensitive code.
- NEXT** `/oto-code-review-fix` to auto-clear the fixable findings.
- RUNTIME** any runtime

/oto-code-review-fix

CODEX

Auto-fix issues found by code review, committing each fix atomically.

- WHEN** After `/oto-code-review` produces a `REVIEW.md`. Spawns the fixer; `--all` processes every finding, `--auto` skips confirmation.
- PRODUCES** Atomic commits (one per fix) · `REVIEW-FIX.md` summarizing what was changed.
- WHY** Applying a known fix to a known issue is execution against a checklist — the diagnosis already happened in code-review. Codex.
- NEXT** Re-run `/oto-code-review` to confirm the findings cleared.
- RUNTIME** any runtime

/oto-audit-fix

SPLIT

Autonomous audit-to-fix pipeline — find, classify, fix, test, commit.

- WHEN** When an audit (audit-uat by default) has surfaced a batch of issues you want cleared without hand-holding. Classifies findings auto-fixable vs manual-only, then fixes the auto ones with test verification. `--max N`, `--severity`, `--dry-run`.
- PRODUCES** Atomic commits for each auto-fix · a classification table separating auto-fixed from manual-only items.
- WHY** Classification is judgment (is this safe to auto-fix?) — Opus; the fixing itself is execution — Codex. The pipeline splits the two internally.
- HEADS UP** Start with `--dry-run` to see the classification before anything is touched. Manual-only items still need you.
- NEXT** Hand the manual-only bucket to `/oto-debug` or `/oto-plan-phase`.
- RUNTIME** any runtime

/oto-pr-branch

CODEX

Create a clean PR branch by filtering out .oto/ commits.

- WHEN** Before opening a PR for human review. Reviewers want code changes, not your planning artifacts cluttering the diff. Pass a target branch (default main).
- PRODUCES** A new branch with .oto/ commits filtered out, ready to PR.
- WHY** Mechanical git assembly — Codex; no judgment involved.
- NEXT** `/oto-ship N`, or open the PR manually with gh.
- RUNTIME** any runtime

/oto-docs-update

CODEX

Generate or update project documentation verified against the codebase.

- WHEN** When docs have drifted from the code. Writes up to 9 doc types, each by an agent that explores the codebase directly — no hallucinated paths or stale signatures. `--force` regenerates all; `--verify-only` checks without writing.
- PRODUCES** Up to 9 documentation files, grounded in the actual code.
- WHY** Doc generation is structured writing anchored to real source — Codex, with the verification step keeping it honest.
- HEADS UP** Flags are opt-in: a documented flag is active only when its literal token appears in the arguments.
- RUNTIME** any runtime

/oto-add-tests

CODEX

Generate tests for a completed phase from UAT criteria and implementation.

- WHEN** After execute-phase when coverage is thin, or after validate-phase flags gaps. Pass extra instructions to steer what gets covered.
- PRODUCES** Test files added to the phase, covering the missing requirements.
- WHY** Generating tests against existing code and stated criteria is pattern matching against requirements — Codex.
- RUNTIME** any runtime

04 DIAGNOSTICS & DEBUG

Pattern recognition, forensics, codebase comprehension. Mostly Opus territory — where reasoning depth pays the highest dividend.

/oto-debug

SPLIT

Systematic debugging with persistent state across context resets.

- WHEN** Any bug that isn't trivially fixable in one session — especially failures whose cause spans several layers. `--diagnose` for the analysis phase; `list / status / continue` manage multi-session bug threads.
- PRODUCES** Persistent debug state under `.oto/bugs/` — hypotheses, evidence, fix attempts, outcome.
- WHY** Diagnosis (why it happens, which layer introduces it, which intervention tests cleanly) is Opus work; the implementation once the cause is known is Codex execution. The textbook case for not compressing both into one engine.
- HEADS UP** Keep the diagnosis in Claude; hand the fix to a Codex session once the cause is pinned.
- NEXT** Once the cause is known, hand the fix to `/oto-execute-phase` or a Codex session.
- RUNTIME** Claude Code to diagnose · Codex to fix

/oto-forensics

OPUS

Post-mortem investigation of failed or stuck oto workflows.

- WHEN** When something went wrong and you don't know what — a stuck phase, missing artifacts, abandoned work, a crashed session. Investigates git history, `STATE.md` anomalies, and artifact integrity.
- PRODUCES** `.oto/forensics/` report with anomaly findings across multiple dimensions (stuck loop, missing artifacts, abandoned work, crash/interruption). Offers to open a GitHub issue for actionable findings.
- WHY** Pattern recognition under uncertainty across heterogeneous evidence — exactly the cross-cutting diagnostic work where Opus's depth pays off.
- NEXT** `/oto-health --repair` for recoverable issues · `/oto-debug` for a specific bug.
- RUNTIME** any runtime

/oto-map-codebase

OPUS

Analyze an existing codebase with parallel mapper agents (brownfield onboarding).

- WHEN** First command on any brownfield project, or when onboarding someone onto an existing codebase. Pass an area to scope it (map-codebase auth).
- PRODUCES** `.oto/codebase/` analysis — module structure, key abstractions, conventions, gotchas.
- WHY** Codebase comprehension is diagnostic work, and downstream planning rides on it — so getting it right compounds. Opus for the read; Gemini's long context helps on a large repo.
- NEXT** `/oto-new-project --auto` using the mapping, or `/oto-discuss-phase` if the project already exists.
- RUNTIME** Claude Code · Gemini for large corpora

/oto-scan

CODEX

Rapid single-area codebase assessment — lightweight map-codebase.

- WHEN** When you need a fast read on one area, not a full map. Spawns one mapper instead of four. `--focus` tech|arch|quality|concerns|tech+arch.
- PRODUCES** Targeted documents under `.oto/codebase/` for the scanned area.
- WHY** Single-area structured extraction where speed matters — Codex does the fast pass; reach for `/oto-map-codebase` and Opus when one area isn't enough.
- NEXT** `/oto-map-codebase` for the full parallel treatment when one area isn't enough.
- RUNTIME** any runtime

05 IDEATION & DISCOVERY

oto's early-stage thinking tools — explore, spike, sketch — for the fuzzy front end before a phase is even a phase.

/oto-explore

OPUS

Socratic ideation and idea routing — think before committing to plans.

- WHEN** When an idea is still fuzzy and you want to think it through out loud before it becomes work. Optionally pass a topic. Probes with questions, optionally spawns research, then routes outputs to the right artifact.
- PRODUCES** Routed outputs — notes, todos, seeds, research questions, requirements, or new phases — depending on where the idea lands.
- WHY** Open-ended ideation and good routing decisions are judgment work — Opus. The value is in the questions asked, not the volume of output.
- NEXT** Wherever it routed — often `/oto-plant-seed`, `/oto-add-todo`, or `/oto-spec-phase`.
- RUNTIME** any runtime

/oto-spike**SPLIT**

Spike an idea through experiential exploration — build to learn.

- WHEN** When feasibility is unknown and you need to build a throwaway experiment to feel the pieces. Idea mode (describe an idea) or frontier mode (no argument — analyzes your spike landscape and proposes next spikes). Auto-creates `.oto/spikes/`.
- PRODUCES** Spike experiments and verified findings under `.oto/spikes/`, integrated with oto's commit and handoff patterns.
- WHY** Deciding what to spike and interpreting the result is Opus reasoning; building the throwaway experiment is Codex execution — hence the split.
- HEADS UP** Spikes are throwaway by design. Capture the learning with `/oto-spike-wrap-up`; don't promote spike code straight to production.
- NEXT** `/oto-spike-wrap-up` to package the findings into a durable skill.
- RUNTIME** Claude Code · Codex to build

/oto-spike-wrap-up**CODEX**

Package spike findings into a persistent project skill.

- WHEN** After one or more spikes, when the verified knowledge should survive the throwaway code and inform the real build.
- PRODUCES** A project skill capturing the spike's validated patterns and data shapes, discoverable in future build conversations.
- WHY** Distilling messy experiment notes into a clean, reusable skill is structured synthesis — Codex.
- NEXT** Reference the new skill from `/oto-spec-phase` or `/oto-plan-phase` on the real build.
- RUNTIME** any runtime

/oto-sketch**OPUS**

Sketch UI/design ideas with throwaway HTML mockups.

- WHEN** When a design direction needs to be seen before it's committed. Produces 2–3 variants per sketch for comparison; loads spike findings to ground mockups in real data shapes. Idea mode or frontier mode. `--quick` for a faster pass.
- PRODUCES** Throwaway HTML mockup variants under `.oto/sketches/`, integrated with oto's commit and handoff patterns.
- WHY** Generating distinct, coherent design directions is exactly the visual-composition judgment Opus is best at.
- NEXT** `/oto-sketch-wrap-up` to lock the chosen direction, then `/oto-ui-phase`.
- RUNTIME** any runtime

/oto-sketch-wrap-up**CODEX**

Package sketch design findings into a persistent project skill.

- WHEN** After sketching, when the chosen design direction should ground future build conversations.
- PRODUCES** A project skill capturing the design decisions and patterns, discoverable later.
- WHY** Distilling design exploration into a reusable contract is structured synthesis — Codex.
- NEXT** `/oto-ui-phase` to formalize the contract into a UI-SPEC.
- RUNTIME** any runtime

06 UTILITY

The smaller commands — quick tasks, capture, todos, backlog. Most route to Codex by speed and shape.

/oto-quick

SPLIT

Ad-hoc task with oto guarantees — no full milestone overhead.

- WHEN** Tasks too big for `/oto-fast` but too small for the full discuss → plan → execute loop. Plain quick for execution-shaped work; add `--discuss` `--research` `--validate` (or `--full`) for design-sensitive tasks that still want rigor. list / status / resume manage quick threads.
- PRODUCES** Plan + execution + atomic commits scoped to the single task. With `--full`, also plan-checking and post-execution verification.
- WHY** Plain quick is execution-shaped — Codex. The `--discuss`/`--research`/`--full` flags pull in Opus thinking. The pill follows whichever you invoke.
- HEADS UP** Don't use it for full features — it bypasses the wave parallelization that makes execute-phase efficient. One-shot tasks only.
- RUNTIME** any runtime

/oto-fast

CODEX

Trivial inline task — no subagents, no planning files, no overhead.

- WHEN** Typo fixes, config tweaks, a line in .gitignore, a forgotten commit, a single-line refactor. The whole point is to skip ceremony.
- PRODUCES** A single commit with the change.
- WHY** Speed and triviality — Codex, inline, no orchestration.
- HEADS UP** Not a replacement for `/oto-quick`. If the task needs research, multi-step planning, or verification, step up.
- RUNTIME** any runtime

/oto-autonomous

OPUS

Run all remaining phases autonomously — discuss → plan → execute per phase.

- WHEN** Honestly: rarely. It compresses the plan/execute split that actually matters. The exception is overnight runs on well-specified milestones where the thinking is done and you just want execution to grind through. `--from` / `--to` / `--only` / `--interactive` scope the run.
- PRODUCES** Whatever the autonomous run produces — usually completed phases.
- WHY** If you must use it, Opus keeps plans and execution coherent end to end. The better move is to break it into discrete commands and route each — Opus to plan, Codex to execute — which is what this guide is for.
- HEADS UP** `--from` N starts partway through. Use it if you've already done the early phases by hand.
- RUNTIME** Codex / ChatGPT

/oto-do

EITHER

Route freeform text to the right oto command automatically.

WHEN When you know what you want but not which command does it. Describe it in plain language and oto picks.

PRODUCES Whatever the dispatched command produces.

WHY Pure routing — the engine is whatever the dispatched command needs.

RUNTIME any runtime

/oto-note

CODEX

Zero-friction idea capture — append, list, or promote to todo.

WHEN Mid-session, when an idea surfaces that isn't part of current work. note "text" appends; note list reviews; note promote N converts to a todo. `--global` for cross-project notes.

PRODUCES A note appended to the project or global notes file.

WHY Capture, not creation — Codex; don't spend Opus tokens on it.

NEXT `/oto-note promote N` when an idea matures into an actionable todo.

RUNTIME any runtime

/oto-add-todo

CODEX

Capture an idea or task as a structured todo from conversation context.

WHEN When you have a specific task to return to but don't want to context-switch now. More structured than a note — produces an actual todo.

PRODUCES A todo entry with description in the project's todo system.

WHY Structured capture — orderly output from Codex.

NEXT `/oto-check-todos` to review and pick one up.

RUNTIME any runtime

/oto-check-todos

CODEX

List pending todos and select one to work on.

WHEN Start of a session when you've forgotten what's pending, or when you want a menu of next things. Pass an area filter to scope it.

PRODUCES An interactive list of todos with selection.

WHY Tabular display — a Codex dashboard pattern.

RUNTIME any runtime

/oto-add-backlog

CODEX

Add an idea to the backlog parking lot (999.x numbering).

- WHEN** Ideas bigger than a todo but not ready for the active milestone. 999.x numbering keeps them out of the active sequence while still scaffolding a phase dir for later.
- PRODUCES** A backlog entry under 999.x phase numbers.
- WHY** Capture and organization, not creation — Codex.
- NEXT** `/oto-review-backlog` when you're ready to promote items.
- RUNTIME** any runtime

/oto-review-backlog

CODEX

Review and promote backlog items to the active milestone.

- WHEN** Before new-milestone, or mid-cycle when scope needs a refresh. For each item: promote, keep, or remove.
- PRODUCES** An updated active sequence with promoted items.
- WHY** Sorting and organization; you make the promote/keep/remove calls in the loop. Codex.
- NEXT** `/oto-plan-phase` on newly promoted items.
- RUNTIME** any runtime

/oto-plant-seed

CODEX

Capture a forward-looking idea with trigger conditions.

- WHEN** When an idea is too early for the current milestone but you don't want it lost in a flat backlog. Captures the full why, when to surface, and breadcrumbs — solving the 'one-liner nobody reads' problem.
- PRODUCES** `.oto/seeds/` entry, scanned automatically by `/oto-new-milestone`, which surfaces matches.
- WHY** Structured capture with prompts — Codex handles the form well.
- NEXT** Surfaces automatically at `/oto-new-milestone`.
- RUNTIME** any runtime

/oto-log

CODEX

Capture an ad-hoc work session as a durable, listable log entry.

- WHEN** For debugging, exploration, env fixes, or conversation-only investigations that should be visible later without becoming a full phase. Fire-and-forget by default; start/end bookmark a session. promote <slug> `--to quick|todo` escalates follow-ups.
- PRODUCES** `.oto/logs/` entry with a six-section body drafted from transcript, git diff, status, and log over the bounded range. Surfaces in `/oto-progress` and `/oto-resume-work`.
- WHY** Drafting an orderly record from observable evidence is summarization — Codex. Entries are immutable by design.
- NEXT** `/oto-log` promote <slug> when a log item turns into real work.
- RUNTIME** any runtime

07 NAVIGATION & WORKSTREAMS

Session state, parallel tracks, and isolated workspaces — the coordination layer for multi-phase and multi-repo work.

/oto-progress

CODEX

Show status and next steps — 'where am I, what's next?'

- WHEN** Returning to a project after a break. `--forensic` appends a 6-check integrity audit (STATE consistency, orphaned handoffs, scope drift, blocking todos, uncommitted code).
- PRODUCES** A progress report; with `--forensic`, also the integrity audit.
- WHY** State navigation and reporting — operational, no judgment. Codex.
- NEXT** `/oto-resume-work` to restore full context, or `/oto-next` to continue.
- RUNTIME** any runtime

/oto-resume-work

CODEX

Restore full context from the previous session.

- WHEN** After a context reset, a new session, or returning days/weeks later. Reads the latest `STATE.md` and any pause-work artifacts to rebuild context.
- PRODUCES** Restored session context with relevant artifacts loaded.
- WHY** Mechanical state restoration — Codex.
- RUNTIME** any runtime

/oto-pause-work

CODEX

Save a context handoff when pausing work mid-phase.

- WHEN** Mid-phase, when you have to stop but want to resume exactly where you left off. Creates a continue-here handoff capturing intermediate state.
- PRODUCES** A continue-here.md with state for the next session to pick up.
- WHY** State capture — Codex.
- NEXT** `/oto-resume-work` next session.
- RUNTIME** any runtime

/oto-manager

CODEX

Interactive command center for managing multiple phases from one terminal.

- WHEN** Power-user mode for parallelizing across phases. A dashboard of all phases with status indicators that recommends next actions and dispatches work — discuss runs inline, plan/execute as background agents.
- PRODUCES** An interactive dashboard with phase status and dispatched work.
- WHY** A dashboard pattern — layout and presentation over fresh reasoning. Codex.
- HEADS UP** Built for parallelization. If you're working linearly, `/oto-next` is simpler.
- RUNTIME** any runtime

/oto-workstreams

CODEX

Manage parallel workstreams for concurrent work on different milestone areas.

- WHEN** When the milestone naturally decomposes into independent tracks that can run concurrently. Subcommands: list, create, switch, status, progress, complete, resume.
- PRODUCES** Workstream directories under .oto/ with per-stream state tracking.
- WHY** Operational coordination — multi-stream tracking is a dashboard pattern. Codex.
- HEADS UP** Don't over-decompose. Workstreams pay off only when tracks are genuinely independent; if they share core code, single-stream phases are simpler.
- NEXT** `/oto-workstreams` switch, then normal phase commands within it.
- RUNTIME** any runtime

/oto-help

EITHER

Show available oto commands and the usage guide.

- WHEN** When this PDF isn't handy.
- PRODUCES** The full command reference and quick-start.
- WHY** Lookup.
- RUNTIME** any runtime

/oto-list-workspaces

CODEX

List active oto workspaces and their status.

- WHEN** When you're running isolated workspaces (separate repo copies with their own .oto/) and need to see what's active.
- PRODUCES** A list of workspaces with status.
- WHY** State listing — Codex.
- NEXT** `/oto-new-workspace` to add one · `/oto-remove-workspace` to clean up.
- RUNTIME** any runtime

/oto-new-workspace

EITHER

Create an isolated workspace with repo copies and an independent `.oto/`.

- WHEN** When a track of work needs full isolation — its own repo copies and planning state, separate from the main tree. `--strategy` `worktree|clone`, `--repos`, `--branch`, `--auto`.
- PRODUCES** An isolated workspace with repo copies and an independent .oto/ setup.
- WHY** Mechanical scaffolding of git worktrees/clones and directory state. Cost or speed decides.
- NEXT** `/oto-list-workspaces` to confirm, then work inside it.
- RUNTIME** any runtime

/oto-remove-workspace

EITHER

*Remove an oto workspace and clean up its worktrees.***WHEN** When an isolated workspace is no longer needed. Pass the workspace name.**PRODUCES** The workspace removed and its worktrees cleaned up.**WHY** Mechanical teardown.**RUNTIME** any runtime**08 PROJECT SETUP & MIGRATION***Onboarding existing work into oto — from scattered docs, external plans, or a GSD-era project.***/oto-ingest-docs**

OPUS

*Bootstrap or merge the full .oto/ setup from existing planning docs.***WHEN** When a project already has scattered ADRs, PRDs, SPECS, and DOCs you want consolidated into oto in one pass. `--mode` new bootstraps (PROJECT/REQUIREMENTS/ROADMAP/STATE); `--mode` merge appends, hard-blocking contradictions with locked decisions. Classifies each doc in parallel.**PRODUCES** A consolidated .oto/ setup · **INGEST-CONFLICTS.md** bucketing auto-resolved, competing-variant, and unresolved-blocker conflicts. The blocker gate prevents writes while contradictions exist.**WHY** Synthesizing heterogeneous docs into one coherent context — and adjudicating conflicts via the ADR > SPEC > PRD > DOC precedence rule — is judgment work. Opus, with Gemini's long context for large doc sets (cap 50 docs/run).**HEADS UP** Resolve everything in the unresolved-blockers bucket before anything is written. Conflicts aren't auto-guessed past the precedence rule.**NEXT** `/oto-new-project` review, or `/oto-discuss-phase` if .oto/ already existed.**RUNTIME** Claude Code · Gemini for large corpora**/oto-import**

CODEX

*Ingest an external plan with conflict detection before writing anything.***WHEN** When you have a single external plan file to bring in. `--from` <filepath> detects conflicts against **PROJECT.md** decisions, writes it as an oto **PLAN.md**, and validates via the plan-checker.**PRODUCES** An oto **PLAN.md** plus a conflict report against locked decisions.**WHY** Translating an external plan into oto's format with a conflict pass is structured conversion against a known schema — Codex.**NEXT** `/oto-execute-phase` once the imported plan is validated.**RUNTIME** any runtime

/oto-migrate

EITHER

Convert a GSD-era project's planning artifacts to oto's command surface.

- WHEN** When a project was set up by Get Shit Done and its artifacts and instruction files still speak GSD. Dry-run by default — produces a classified report and writes nothing; pass `--apply` to write. Idempotent; refuses a half-migrated tree unless `--force`.
- PRODUCES** Migrated planning state, roadmap, phase dirs, and CLAUDE/AGENTS/GEMINI files · a timestamped backup unless `--no-backup`.
- WHY** A project-level converter, not an installer. The rename map carries the logic, so it's mechanical transformation — but always preview the dry run first.
- HEADS UP** It touches your project tree, not your runtime config dirs (those stay owned by oto install). Review the dry-run report before `--apply`.
- NEXT** `/oto-progress` to confirm oto reads the migrated state cleanly.
- RUNTIME** any runtime

09 CONFIG & UPDATE

Setup and maintenance. Run once, mostly forget — plus the housekeeping that keeps .oto/ healthy.

/oto-settings

EITHER

Configure oto workflow toggles and model profile interactively.

- WHEN** First-time setup, or when changing defaults — context-coverage gates, plan bounce passes, parallelization, workflow mode, model profile.
- PRODUCES** Updated `.oto/config.json` (and `~/.oto/defaults.json` for profile).
- WHY** A config menu — the engine barely matters here.
- NEXT** `/oto-set-profile` to change model assignments directly · `/oto-settings-advanced` for power knobs.
- RUNTIME** any runtime

/oto-settings-advanced

EITHER

Power-user configuration — plan bounce, timeouts, cross-AI execution, runtime knobs.

- WHEN** When the basic settings aren't enough. Sectioned prompts cover planning/execution/discussion tuning, cross-AI execution, git customization, and runtime/output.
- PRODUCES** Updated advanced config in `.oto/config.json`.
- WHY** A config menu. This is where you wire execution to Codex (cross-AI execution) so the CODEX-tagged commands actually run on GPT.
- RUNTIME** any runtime

/oto-settings-integrations

EITHER

*Configure third-party API keys, review CLI routing, and skill injection.***WHEN** When wiring oto to external services — API keys, which CLI handles code review, agent-skill injection.**PRODUCES** Updated integration config.**WHY** A config menu.**RUNTIME** any runtime**/oto-set-profile**

EITHER

*Switch model profile for oto agents (quality / balanced / budget / inherit).***WHEN** Switching cost mode for oto's Claude-side agents, or pointing them at your session model. Use inherit on non-Anthropic providers to avoid surprise Anthropic calls.**PRODUCES** Updated config with the new profile.**WHY** A config command. It tunes which Claude model backs each agent — orthogonal to the Opus-vs-Codex routing this guide describes, which is about where the thinking versus the execution happens.**HEADS UP** On non-Claude runtimes the installer omits model IDs, so each agent uses the runtime's default model; add model_overrides to assign per-agent models.**RUNTIME** any runtime**/oto-sync-skills**

EITHER

*Sync managed oto skills across runtime roots.***WHEN** After an update, so multi-runtime users (Claude Code / Codex / Gemini) stay aligned and don't drift between roots.**PRODUCES** Managed skills synced across the installed runtime roots.**WHY** Mechanical file sync.**RUNTIME** any runtime**/oto-update**

EITHER

*Update oto to the latest version with changelog display.***WHEN** Periodically. Shows the changelog before installing so you can decide whether to take the update.**PRODUCES** An updated oto installation.**WHY** Mechanical install.**NEXT** **/oto-sync-skills** if you run multiple runtimes.**RUNTIME** any runtime

/oto-stats

CODEX

Display project statistics — phases, plans, requirements, git metrics, timeline.

- WHEN** A quick health check — phases done, commits, time on phase, token usage. Useful before a status update or deciding whether to keep grinding.
- PRODUCES** A project metrics dashboard in the terminal.
- WHY** A dashboard — cleaner layouts come from Codex's presentation strength, not deep reasoning.
- RUNTIME** any runtime

/oto-health

CODEX

Diagnose .oto/ directory integrity and optionally repair.

- WHEN** When something feels off — missing files, broken references, weird state. `--repair` auto-fixes recoverable issues.
- PRODUCES** A health report with issues flagged; with `--repair`, fixes applied.
- WHY** Linter work — comparing actual state against the expected schema. Codex.
- NEXT** `/oto-forensics` if `--repair` can't resolve it.
- RUNTIME** any runtime

/oto-cleanup

CODEX

Archive accumulated phase directories from completed milestones.

- WHEN** .oto/ has gotten cluttered with old phase dirs from completed milestones. Run periodically to keep it navigable.
- PRODUCES** Old phase directories moved into an archive structure.
- WHY** Mechanical housekeeping — Codex.
- RUNTIME** any runtime

/oto-undo

EITHER

Safe git revert — roll back phase or plan commits with dependency checks.

- WHEN** When a phase or plan needs rolling back. `--last` N for interactive selection, `--phase` NN for a whole phase, `--plan` NN-MM for one plan. Uses the phase manifest and gates on confirmation before executing.
- PRODUCES** The selected commits reverted, with a dependency check first.
- WHY** Mechanical git revert — the dependency check and confirmation gate keep it safe. Cost or speed decides the engine.
- HEADS UP** It reverts via git using the phase manifest. Check the dependency warning before confirming so you don't orphan dependent work.
- RUNTIME** any runtime

The Canonical Loop

The default sequence for any non-trivial phase. Steps 1–3 are skippable when the work is crisp or backend-only; the cross-AI review at step 5 and the handoff to Codex at step 6 are what actually matter.

1	/oto-spec-phase Lock what/why	OPUS
2	/oto-discuss-phase Surface trade-offs	OPUS
3	/oto-ui-phase Lock design contract	OPUS
4	/oto-plan-phase Push toward spec-style plan	OPUS
5	/oto-review --all Cross-pollinate before exec	SPLIT
6	/oto-execute-phase Switch to Codex / GPT	CODEX
7	/oto-verify-work Fresh-eyes UAT	OPUS
8	/oto-ui-review Visual audit	OPUS
9	/oto-ship Auto-generate PR	CODEX

The step 5 → 6 handoff. Before execution, get the plan into spec shape: numbered steps, explicit file paths, acceptance criteria per step, no narrative. That legibility — not the engine — is what makes the Opus → Codex handoff land. Push **/oto-plan-phase** toward that shape, then run the wave in Codex.

Three Rules That Override the Pills

RULE 1 **Opus plans, Codex executes.** The split is the point — a strong model writing a clean spec-style plan that a hard executor grinds out beats either engine doing both. The pills route accordingly: Opus for thinking, Codex for execution.

RULE 2 **Codex execution is opt-in.** The **CODEX** pill is a recommendation, not automatic. To actually run a command on GPT, run oto inside a Codex session or enable cross-AI execution in **/oto-settings-advanced**; otherwise it runs in Claude Code.

RULE 3

Cross-AI review is multi-model on purpose. `/oto-review` and `/oto-plan-review-convergence` spawn external CLIs (Codex, Gemini, Claude). Their value is that each engine has different blind spots — never collapse them into a single reviewer.

Built for the oto hybrid framework (v0.3.0) — GSD's spec-driven workflow plus Superpowers' capabilities under one `/oto-*` surface across Claude Code, Codex, and Gemini CLI. Engine assignments follow the Opus-for-thinking / Codex-for-execution pattern; they are guidance, not hard wiring — your runtime and `/oto-set-profile` settings always win. Regenerate this guide when commands are added.